

Real-world (rebuild a Project 1 feature, LCEL-style) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (comparing more than just correctness). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The mapping, and the two deliverables](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The mapping, and the two deliverables

Basic Version

You already built a structured-extraction feature in Project 1, using the raw OpenAI SDK. This exercise is not about changing that feature's job — it's about writing the *same* feature a second way, with LCEL, and keeping both.

Go find Project 1's structured-extraction code first. Read it again before writing anything new — you need to know exactly what input it takes and what output it returns, so your LCEL version matches.

Then think about how each raw-SDK step maps onto an LCEL piece: building messages → a prompt template; calling the API → `ChatOpenAI` ; reading `response_format` 's result → a structured output parser.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

This exercise has two deliverables: a new LCEL version of Project 1's structured-extraction feature (in a new file, not replacing the original), and a small script that runs both versions on the same 3 inputs and diffs the results.

The mapping from raw SDK to LCEL: your `messages` list becomes a `ChatPromptTemplate`; your `client.chat.completions.create(...)` call becomes a `ChatOpenAI` instance wired into a chain; your `response_format=YourPydanticModel` becomes `.with_structured_output(YourPydanticModel)`.

Reuse the same Pydantic model Project 1 already defines for its structured output — don't redefine it, import it; this is exactly what proves both versions produce the *same shape*, not just a similar-looking one. Keep `temperature=0` on both versions so any difference you see is from the method, not from randomness. Two Pydantic model instances with the same field values are equal with `==` by default — use that directly instead of writing manual field comparisons.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

"Both versions give the same answer" is only half the comparison this document's Core Concepts promised — the other half is the trade-off itself: LangChain adds a layer between your code and the actual API call. That layer has to cost *something*, even if correctness is identical. The real design question: **is that cost visible, and is it worth it?**

Two things are worth actually measuring, not assuming:

- **Latency** — does the LCEL version take noticeably longer per call than the raw SDK version, on the same input? (Usually: a small, close-to-negligible amount, from the extra Python-level indirection — worth confirming for yourself rather than assuming either "no difference" or "much slower.")
- **Token usage** — do the two versions send the *exact* same prompt to the model, or does LangChain's prompt template add any wrapping text of its own that changes the token count, and therefore the cost, even when the final answer matches?

```
import time

start = time.perf_counter()
raw_result = extract_raw(text)
raw_seconds = time.perf_counter() - start

start = time.perf_counter()
lcel_result = lcel_chain.invoke({"text": text})
lcel_seconds = time.perf_counter() - start
```

Sketch how you'd add this timing to your comparison script, and what you'd conclude if the LCEL version were, say, 20% slower — is that a problem, or a reasonable cost for the reuse LangChain gives you? Write your answer down before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both prove the two versions produce the same *answer*. Advanced asks whether they cost the same to get there — timing each call and comparing token usage — which is the concrete, measured version of the "what does this layer cost" question Doc05's own Core Concepts section raises but never actually measures for you.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
import Project 1's raw extraction function and its Pydantic model

build an LCEL chain using the same Pydantic model

make a list of 3 test inputs

for each input:
    run the raw version -> result A
    run the LCEL version -> result B
    print both
    print whether they match
```

Here is almost the whole thing — fill in your own project's real names:

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from project_1 import extract_raw, ExtractedData    # your real Project 1 names

prompt = ChatPromptTemplate.from_template("Extract structured data from: {text}")
lcel_chain = prompt | ChatOpenAI().with_structured_output(ExtractedData)
```

Expected output if you run just this: nothing yet — add the loop over your 3 test inputs, calling both `extract_raw` and `lcel_chain.invoke` .

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
def build_lcel_chain(model_name="gpt-4o-mini"):
    prompt = ChatPromptTemplate.from_template("...")
    return prompt | ChatOpenAI(model=model_name, temperature=0).with_structured_output(...)

test_inputs = [input_1, input_2, input_3]

for text in test_inputs:
    raw_result = extract_raw(text)
    lcel_result = lcel_chain.invoke({"text": text})
    match = raw_result == lcel_result
    print(f"Input: {text}")
    print(f"  raw: {raw_result}")
    print(f"  lcel: {lcel_result}")
    print(f"  match: {match}")
```

Write the full comparison script (the loop, printing both results and whether they match) yourself before checking Hint 3.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
for each input:
    time the raw call
    time the lcel call
    compare results (as before)
    compare the two timings

after the loop:
    print the average raw time and average lcel time
```

Turning that into real code — fill in the missing piece yourself:

```
import time
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from project_1 import extract_raw, ExtractedData

prompt = ChatPromptTemplate.from_template("Extract structured data from: {text}")
lcel_chain = prompt | ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output(ExtractedData)

test_inputs = ["input one text", "input two text", "input three text"]
raw_times = []
lcel_times = []

for text in test_inputs:
    start = time.perf_counter()
    raw_result = extract_raw(text)
    raw_times.append(time.perf_counter() - start)

    start = time.perf_counter()
    lcel_result = lcel_chain.invoke({"text": text})
    lcel_times.append(time.perf_counter() - start)

    print(f"{text!r} -> match: {raw_result == lcel_result}")

# your turn: after the loop, print the average of raw_times and
# the average of lcel_times, so the timing difference (if any) is
# a number you can actually read, not something you have to eyeball
...
```

Expected output: a match line per input, plus 2 average-timing lines at the end — confirm whether the LCEL version is meaningfully slower, about the same, or (less commonly) faster.

Fill in the averaging yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both compare *what* the two versions return. Advanced adds a second axis to the comparison — *how long* each version takes — and reports it as real, averaged numbers instead of leaving "LangChain adds overhead" as an unverified claim.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)